



**UNIVERSIDAD ESPÍRITU SANTO
FACULTAD DE INGENIERÍA**

Tema:

Ae1 | Informe comparativo de modelos de proceso

Nombre:

Anthony Patricio Morocho Castillo

Profesor:

Ph.D. Jaime Paul Sayago Heredia

Asignatura:

Ingeniería de software I – **Código:** UCOM304

Fecha de entrega:

24/08/2026

1. Introducción

El desarrollo de software requiere mucho más que escribir código. Para construir una solución que responda correctamente a las necesidades de los usuarios es necesario organizar actividades como el levantamiento de requisitos, el diseño, la implementación, las pruebas, la entrega y el mantenimiento. Los modelos de proceso proporcionan una estructura que permite ordenar estas actividades y definir de qué manera se avanza durante el desarrollo de un proyecto.

La selección del modelo de proceso es importante porque no todos los proyectos presentan las mismas condiciones. Algunos poseen requisitos relativamente estables desde el comienzo, mientras que otros necesitan incorporar cambios frecuentes o validar las necesidades mediante la participación constante de los usuarios. También existen proyectos donde el riesgo técnico o empresarial es elevado y debe ser analizado antes de continuar con el desarrollo. Por esta razón, no existe un modelo universalmente adecuado para todos los casos.

El propósito de este informe es comparar los modelos en cascada, incremental, de prototipos y en espiral, considerando su organización, capacidad para gestionar cambios, participación de stakeholders, tratamiento del riesgo, forma de entrega, ventajas, limitaciones y contextos recomendados. Posteriormente, estos criterios serán aplicados al caso de un Sistema de Gestión de Tutorías Académicas para seleccionar el modelo de proceso que mejor responda a sus características.

La decisión final se sustentará principalmente en cuatro elementos: el nivel de riesgo del proyecto, la posibilidad de cambios en los requisitos, las características del equipo de desarrollo y la criticidad del sistema. De esta manera, la selección del modelo será resultado del análisis del contexto y no únicamente de una preferencia metodológica.

2. Marco conceptual

2.1 Ciclo de vida del software

El ciclo de vida del software comprende las distintas etapas por las que pasa un producto desde que surge la necesidad de desarrollarlo hasta su operación, mantenimiento y eventual retiro. Estas etapas pueden incluir actividades relacionadas con requisitos, diseño, implementación, pruebas, despliegue, mantenimiento y evolución.

La norma ISO/IEC/IEEE 12207 establece un marco de referencia para los procesos del ciclo de vida del software y permite organizar las actividades necesarias durante la adquisición, desarrollo, operación y mantenimiento de productos y servicios de software (ISO/IEC/IEEE, 2017).

El ciclo de vida no determina necesariamente que todas las actividades deban ejecutarse una sola vez o de manera estrictamente secuencial. La forma en que estas actividades se organizan depende del modelo de proceso seleccionado.

2.2 Proceso de software

Un proceso de software puede entenderse como el conjunto organizado de actividades, métodos, prácticas y tareas utilizadas para desarrollar y mantener un producto de software. Sommerville (2016) plantea que, aunque los procesos pueden variar considerablemente entre organizaciones, normalmente incluyen actividades

fundamentales relacionadas con la especificación, desarrollo, validación y evolución del software.

Por lo tanto, el proceso permite definir qué debe realizarse, en qué momento, quién participa y qué productos o resultados deben generarse durante el proyecto.

2.3 Modelo de proceso

Un modelo de proceso representa de forma simplificada la manera en que se organiza el desarrollo de software. Su objetivo es proporcionar una estructura que facilite la planificación y coordinación de las actividades del proyecto.

Pressman y Maxim (2020) explican que los modelos de proceso permiten establecer un marco de trabajo para desarrollar software de manera organizada. Dependiendo de las características del proyecto, las actividades pueden ejecutarse secuencialmente, mediante incrementos, mediante experimentación con prototipos o utilizando ciclos orientados a la identificación y reducción de riesgos.

Por este motivo, elegir un modelo no consiste únicamente en seleccionar una metodología conocida, sino en determinar cuál estructura responde mejor a las condiciones reales del proyecto.

2.4 Enfoque predictivo

Un enfoque predictivo busca definir desde las primeras etapas una cantidad considerable de información sobre el proyecto, incluyendo requisitos, planificación, recursos y resultados esperados. Posteriormente, el desarrollo intenta seguir dicha planificación.

Este enfoque resulta conveniente cuando existe una elevada estabilidad en los requisitos y es posible conocer con suficiente precisión qué debe construirse. El modelo en cascada constituye uno de los ejemplos más representativos de este enfoque.

Sin embargo, cuando existen cambios frecuentes, la modificación de decisiones tomadas previamente puede generar costos adicionales debido a la necesidad de regresar a etapas que ya habían sido consideradas finalizadas.

2.5 Enfoque adaptativo

Los enfoques adaptativos aceptan que parte del conocimiento necesario para desarrollar el producto aparecerá durante el proyecto. En lugar de intentar establecer desde el inicio todos los detalles, permiten realizar entregas, obtener retroalimentación y ajustar progresivamente la solución.

Modelos como el incremental y el uso de prototipos presentan características compatibles con esta forma de trabajo. El modelo en espiral también incorpora ciclos sucesivos, aunque su principal elemento diferenciador es la gestión explícita del riesgo.

3. Comparación de modelos

3.1 Modelo en cascada

El modelo en cascada organiza el desarrollo mediante fases que avanzan principalmente de forma secuencial. Normalmente se inicia con la definición de requisitos y posteriormente se continúa con diseño, implementación, pruebas y entrega.

Su principal ventaja es la claridad de la planificación y de los entregables asociados a cada etapa. También facilita la documentación y el seguimiento cuando los requisitos son conocidos y permanecen relativamente estables.

Sin embargo, presenta dificultades cuando los requisitos cambian en etapas avanzadas. Una modificación importante puede obligar a revisar documentación, diseños y componentes ya desarrollados. Además, el usuario puede esperar hasta fases posteriores para observar un producto funcional.

Por estas razones, resulta más apropiado en proyectos con requisitos bien establecidos, cambios limitados y necesidad de una planificación y documentación formal.

3.2 Modelo incremental

El modelo incremental divide el producto en diferentes entregas funcionales. En lugar de construir todo el sistema antes de mostrar resultados, se desarrollan primero las funcionalidades prioritarias y posteriormente se incorporan nuevos componentes mediante incrementos sucesivos.

Una ventaja importante es que los usuarios pueden utilizar o evaluar partes funcionales del sistema antes de completar todo el proyecto. La retroalimentación obtenida puede utilizarse para modificar prioridades o mejorar los siguientes incrementos.

Este modelo también reduce el impacto de determinados cambios porque no todo el sistema se encuentra desarrollado al mismo tiempo. Sin embargo, necesita una arquitectura suficientemente sólida para permitir la integración progresiva de las funcionalidades.

También requiere una adecuada priorización. Si los incrementos no se planifican correctamente, podrían aparecer dependencias entre funcionalidades o problemas de integración.

3.3 Modelo de prototipos

El modelo de prototipos se utiliza principalmente cuando existen dudas sobre las necesidades de los usuarios, la interfaz o determinadas características de la solución. Se construye una representación parcial del sistema para obtener información y retroalimentación antes de desarrollar completamente el producto.

Los prototipos pueden ser descartables, cuando solamente se utilizan para comprender requisitos y luego se reemplazan, o evolutivos, cuando la solución se mejora progresivamente hasta transformarse en parte del producto final.

Su fortaleza principal es reducir la incertidumbre. Los usuarios pueden observar una representación de la solución y expresar con mayor precisión qué necesitan y qué modificaciones deberían realizarse.

Entre sus riesgos se encuentra que los usuarios interpreten un prototipo como si fuera el producto terminado. Además, cuando un prototipo construido rápidamente se transforma directamente en el producto final sin revisar su calidad interna, pueden aparecer problemas de arquitectura y mantenimiento.

3.4 Modelo en espiral

El modelo en espiral propuesto por Boehm se caracteriza por organizar el desarrollo mediante ciclos sucesivos fuertemente orientados al análisis y reducción del riesgo. En cada ciclo se establecen objetivos, se estudian alternativas, se analizan riesgos y se planifica la siguiente etapa (Boehm, 1988).

El riesgo determina qué actividades deben realizarse a continuación. Por ejemplo, cuando existe incertidumbre sobre una tecnología, puede desarrollarse una prueba o prototipo antes de comprometer una inversión mayor.

Este enfoque resulta especialmente útil en sistemas complejos, costosos o con consecuencias importantes en caso de fallo. Sin embargo, requiere experiencia para identificar y analizar correctamente los riesgos. Su aplicación también puede resultar innecesariamente compleja para sistemas relativamente pequeños y de criticidad moderada.

3.5 Matriz comparativa

Criterio	Cascada	Incremental	Prototipos	Espiral
Organización	Fases principalmente secuenciales con planificación inicial detallada.	Desarrollo dividido en incrementos funcionales priorizados.	Construcción y evaluación sucesiva de representaciones parciales del sistema.	Ciclos iterativos determinados por objetivos y riesgos.
Gestión del cambio	El cambio tardío puede ser costoso porque afecta fases ya terminadas.	Permite incorporar cambios entre incrementos con menor impacto.	Facilita modificar requisitos después de observar y evaluar el prototipo.	Los cambios pueden analizarse en cada ciclo junto con sus riesgos.
Stakeholders	Participan principalmente durante requisitos y aceptación.	Participan frecuentemente evaluando las entregas parciales.	Participación elevada para evaluar y mejorar el prototipo.	Participan en evaluaciones y decisiones asociadas a cada ciclo.
Gestión del riesgo	El riesgo no constituye el elemento principal del modelo.	Puede reducir riesgo mediante entregas tempranas,	Reduce principalmente riesgos relacionados	La identificación y reducción del riesgo constituye su

Criterio	Cascada	Incremental	Prototipos	Espiral
		pero no posee un proceso de riesgo explícito.	con requisitos e interacción.	característica central.
Entrega	El producto funcional suele obtenerse principalmente al finalizar el desarrollo.	Se obtienen versiones funcionales progresivamente.	Se obtienen prototipos tempranos que pueden o no convertirse en producto final.	Cada ciclo produce mayor conocimiento y puede generar versiones progresivas.
Ventajas	Claridad, documentación, planificación y facilidad de seguimiento.	Valor temprano, flexibilidad, retroalimentación y priorización.	Clarificación de requisitos y validación de interfaces y necesidades.	Gestión sistemática de riesgos y capacidad para abordar proyectos complejos.
Limitaciones	Baja flexibilidad frente a cambios y validación funcional tardía.	Requiere buena arquitectura, priorización e integración continua.	Puede generar expectativas incorrectas y deuda técnica si se evoluciona sin control.	Mayor costo, complejidad y necesidad de experiencia en gestión de riesgos.
Contexto recomendado	Requisitos estables y proyectos con fuerte necesidad de documentación y control.	Sistemas donde pueden entregarse funcionalidades gradualmente y existe retroalimentación frecuente.	Proyectos con requisitos o interfaz poco claros.	Proyectos grandes, complejos, costosos o de alto riesgo.

3.6 Análisis comparativos

Las diferencias entre los modelos muestran que la decisión no debe realizarse considerando únicamente cuál permite desarrollar más rápido. Cada modelo responde a problemas diferentes.

Cascada ofrece una estructura fácil de comprender y controlar cuando existe estabilidad. Su principal dificultad aparece cuando el equipo descubre durante el desarrollo que una necesidad fue interpretada incorrectamente. Debido a la secuencia de fases, corregir esta situación puede afectar actividades previamente completadas.

El modelo incremental presenta una diferencia importante: no necesita esperar hasta que todo el sistema esté terminado para generar valor. Una primera versión puede resolver algunas necesidades esenciales mientras otras permanecen en desarrollo.

Esto permite obtener evidencia real sobre el funcionamiento de la solución y ajustar los siguientes incrementos.

El modelo de prototipos también proporciona resultados tempranos, pero su objetivo puede ser diferente. Mientras que un incremento normalmente pretende convertirse en una parte funcional del producto, un prototipo puede construirse solamente para aprender y resolver incertidumbre. Por ello resulta especialmente útil cuando el equipo necesita comprender una interfaz, flujo o requisito antes de desarrollar la solución definitiva.

Finalmente, el modelo en espiral incorpora elementos iterativos, pero establece el riesgo como criterio fundamental para determinar el siguiente paso. Boehm (1988) describe precisamente el enfoque como un modelo orientado al riesgo. Esta característica puede justificar su complejidad en proyectos donde una decisión equivocada tendría consecuencias económicas, técnicas o de seguridad significativas.

En consecuencia, los cuatro modelos poseen utilidad. La selección depende de la relación entre las características del proyecto y las fortalezas del proceso elegido.

4. Aplicación al caso: Sistema de gestión de tutorías académicas

El proyecto seleccionado consiste en desarrollar un Sistema de Gestión de Tutorías Académicas para una institución educativa. Su objetivo es centralizar la publicación de horarios, reserva y cancelación de tutorías, confirmaciones, seguimiento de sesiones y generación de información para docentes y personal administrativo.

Actualmente, estas actividades pueden realizarse mediante diferentes canales y procedimientos manuales, generando problemas como duplicación de reservas, dificultad para consultar disponibilidad y falta de seguimiento de las tutorías.

Elemento	Descripción del caso
Contexto	Institución educativa que necesita digitalizar y centralizar el proceso de gestión de tutorías académicas.
Usuarios stakeholders y	Estudiantes, docentes, coordinadores académicos, personal administrativo y personal responsable del sistema.
Incertidumbre	Moderada. Algunas reglas pueden definirse inicialmente, pero la experiencia de estudiantes y docentes puede revelar necesidades no identificadas durante el levantamiento inicial.
Cambios esperados	Moderados o altos. Pueden cambiar reglas de cancelación, duración de tutorías, horarios, mecanismos de confirmación, notificaciones y reportes.
Riesgos	Doble reserva de horarios, inconsistencias en disponibilidad, pérdida de información, notificaciones incorrectas, acceso no autorizado a datos y baja aceptación por parte de los usuarios.
Criticidad	Media. Los errores afectan procesos académicos y datos de los usuarios, pero el sistema no se considera crítico para la vida o seguridad física.

Características del equipo	Equipo pequeño o mediano que puede coordinar entregas periódicas, dividir funcionalidades e incorporar retroalimentación de usuarios.
-----------------------------------	---

El sistema presenta funcionalidades que pueden separarse de forma relativamente clara. Por ejemplo, inicialmente puede implementarse el acceso de usuarios y publicación de horarios. Posteriormente puede añadirse la reserva de tutorías, cancelación, notificaciones, seguimiento y finalmente reportes administrativos.

Esta característica hace posible entregar valor progresivamente sin esperar al desarrollo de todas las funciones.

Además, la interacción con los usuarios es importante. Aunque inicialmente puedan definirse reglas para reservar una tutoría, es probable que durante las pruebas aparezcan situaciones no consideradas, como límites de reservas por estudiante, tiempos mínimos de cancelación, reprogramaciones, bloqueos de horarios o diferentes políticas según la asignatura.

Por lo tanto, un proceso que permita recibir retroalimentación durante el desarrollo presenta una ventaja considerable.

5. Modelo recomendado

Para el Sistema de Gestión de Tutorías Académicas se recomienda utilizar el modelo incremental.

La selección no se fundamenta únicamente en que el modelo sea flexible. Se basa en la relación entre las características específicas del proyecto y la capacidad del modelo para responder a ellas.

Criterio	Característica del caso	Implicación	Respuesta del modelo incremental
Riesgo	Existe posibilidad de reservas duplicadas, errores en disponibilidad o reglas académicas incorrectamente interpretadas.	Es conveniente detectar problemas antes de construir todo el sistema.	Los primeros incrementos permiten validar las reglas esenciales antes de ampliar la solución.
Cambio	Las políticas de reserva, cancelación y notificación pueden modificarse después de ser utilizadas por estudiantes y docentes.	El proceso debe permitir ajustes sin rediseñar completamente el producto.	Los cambios pueden incorporarse y priorizarse para incrementos posteriores.
Equipo	El proyecto puede ser desarrollado por un equipo pequeño o mediano.	Se necesita una organización que permita dividir el trabajo y generar	Las funcionalidades pueden agruparse en incrementos relativamente

		resultados frecuentes.	pequeños y coordinados.
Criticidad	La criticidad es media y los errores afectan principalmente operaciones académicas.	Se requiere control de calidad, pero no un proceso tan complejo como el utilizado para sistemas de alto riesgo.	El incremental permite validar progresivamente el producto sin incorporar el costo de un proceso de riesgo tan formal como el espiral.

Una posible organización de los incrementos sería la siguiente:

Incremento 1: autenticación, usuarios y perfiles.

Incremento 2: publicación y consulta de horarios disponibles.

Incremento 3: reserva de tutorías y control de duplicidad.

Incremento 4: cancelación, reprogramación y confirmaciones.

Incremento 5: notificaciones y seguimiento de tutorías.

Incremento 6: reportes y funcionalidades administrativas.

Esta organización permitiría que estudiantes, docentes y responsables académicos prueben las funcionalidades principales desde etapas relativamente tempranas. Los problemas detectados durante un incremento podrían considerarse antes de desarrollar los siguientes.

El modelo de prototipos también podría resultar útil, especialmente durante el diseño inicial de las pantallas de reserva y consulta de disponibilidad. Sin embargo, el problema no se limita exclusivamente a descubrir requisitos. El proyecto necesita entregar progresivamente funcionalidades reales, por lo que el incremental ofrece una estructura más completa para organizar todo el desarrollo. Los prototipos podrían utilizarse como una técnica de apoyo dentro de determinados incrementos.

Un enfoque estrictamente en cascada resultaría menos conveniente porque existen requisitos susceptibles de cambio después de que los usuarios interactúen con las primeras versiones. Esperar hasta las últimas etapas para validar el producto aumentaría el costo de corregir interpretaciones incorrectas.

El modelo en espiral permitiría gestionar formalmente los riesgos, pero podría resultar desproporcionado para este proyecto. La criticidad del sistema es moderada y no se identifican inicialmente riesgos técnicos o económicos suficientemente elevados como para justificar la complejidad de ejecutar un proceso completamente orientado al riesgo.

Por estas razones, el modelo incremental proporciona el mejor equilibrio entre capacidad de adaptación, entrega progresiva, participación de los stakeholders y complejidad de gestión para el proyecto analizado.

6. Conclusiones

La comparación demuestra que no existe un modelo de proceso universalmente superior. La conveniencia de cada alternativa depende de características como

estabilidad de requisitos, participación de usuarios, riesgo, criticidad y capacidad del equipo.

El modelo en cascada ofrece una estructura clara y documentada cuando los requisitos son estables, pero pierde efectividad cuando aparecen modificaciones importantes durante etapas avanzadas del proyecto.

Los modelos incrementales y de prototipos proporcionan mecanismos para obtener retroalimentación temprana, aunque persiguen objetivos distintos. El primero busca construir progresivamente partes funcionales del producto, mientras que el segundo se orienta principalmente a reducir incertidumbre y comprender mejor las necesidades.

El modelo en espiral resulta especialmente apropiado cuando el nivel de riesgo justifica realizar análisis explícitos antes de tomar decisiones importantes. Sin embargo, su complejidad puede ser innecesaria en proyectos pequeños o de criticidad moderada.

Para el Sistema de Gestión de Tutorías Académicas, el modelo incremental presenta la mejor relación con las características analizadas. La posibilidad de dividir las funcionalidades, obtener retroalimentación frecuente y responder a cambios permite reducir errores y entregar valor progresivamente sin incorporar una complejidad de gestión desproporcionada.

7. Referencias

Boehm, B. W. (1988). A spiral model of software development and enhancement. *Computer*, 21(5), 61–72. <https://doi.org/10.1109/2.59>

ISO/IEC/IEEE. (2017). *ISO/IEC/IEEE 12207:2017 Systems and software engineering—Software life cycle processes*. International Organization for Standardization.

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

Ficha de fuentes

Fuente	Tipo	Idea principal	Modelo / criterio	Uso en el informe
Sommerville (2016)	Libro académico	Procesos, ciclo de vida y modelos de desarrollo de software.	Marco conceptual y comparación general.	Definición del proceso y fundamento conceptual.
Pressman & Maxim (2020)	Libro académico	Organización del proceso de software y características de	Cascada, incremental y prototipos.	Comparación de fortalezas, limitaciones y aplicación.

		distintos modelos.		
ISO/IEC/IEEE 12207:2017	Estándar internacional	Marco común para los procesos del ciclo de vida del software.	Ciclo de vida y organización de procesos.	Sustento del marco conceptual.
Boehm (1988)	Artículo científico	Desarrollo del modelo espiral como enfoque orientado al riesgo.	Espiral y gestión de riesgo.	Sustento del análisis del modelo en espiral.

Declaración de uso de inteligencia artificial

Campo	Información
Herramienta utilizada	ChatGPT, desarrollado por OpenAI.
Propósito de uso	Apoyo para organizar la estructura del informe, comparar los modelos de proceso, estructurar la matriz comparativa y mejorar la claridad de la redacción.
Prompt o instrucción principal	Analizar las instrucciones de la actividad y elaborar un borrador de informe comparativo sobre los modelos cascada, incremental, prototipos y espiral, aplicándolos al caso de un Sistema de Gestión de Tutorías Académicas.
Resultado utilizado	Borrador de secciones, matriz comparativa, organización del análisis del caso y propuesta de justificación del modelo seleccionado.
Modificaciones realizadas por el estudiante	Revisión del contenido, adaptación de la redacción, comprobación de la relación entre el caso y el modelo seleccionado y ajustes realizados de acuerdo con los contenidos estudiados en la asignatura.
Procedimiento utilizado para verificar la información	Se contrastaron los conceptos con la bibliografía indicada en la actividad, incluyendo Sommerville, Pressman y Maxim, ISO/IEC/IEEE 12207 y el artículo académico de Boehm. Se verificó además que la argumentación relacionara riesgo, cambio, equipo y criticidad con la selección del modelo.